CÓDIGOCLARO · RUTA JAVASCRIPT

Tema 14 · Pruebas y calidad

Guía de estudio · DAM/DAW

Pruebas unitarias, casos límite, lint y funciones puras.

1. Explicación

Una prueba unitaria compara el comportamiento observable con el resultado esperado. Casos límite e inválidos complementan el caso típico. Node incluye test runner node:test y aserciones node:assert/strict; un linter puede señalar patrones de riesgo antes de ejecutar.

2. Ejemplo

Código · javascript

```
import test from "node:test";
import assert from "node:assert/strict";
function esPar(n) { return n % 2 === 0; }
test("esPar distingue par e impar", () => {
  assert.equal(esPar(4), true);
  assert.equal(esPar(3), false);
  assert.equal(esPar(0), true);
});
```

Resultado esperado

```
node --test informa que la prueba pasa.
```

3. Errores frecuentes

- Probar solo el caso típico.
- Hacer que una prueba dependa del orden de ejecución.
- Comprobar detalles internos en lugar del resultado observable.

4. Ejercicio

Prueba máximo de lista no vacía para positivos, negativos y una sola entrada.

Casos que debe superar

Entrada	Resultado esperado
[2,7,1]	7
[-5,-2]	-2
[4]	4

5. Solución explicada

Código · javascript

```
import test from "node:test";
import assert from "node:assert/strict";
function maximo(xs) {
  if (xs.length === 0) throw new RangeError("Lista vacía");
  return Math.max(...xs);
}
test("maximo", () => {
  assert.equal(maximo([2,7,1]), 7);
  assert.equal(maximo([-5,-2]), -2);
  assert.equal(maximo([4]), 4);
});
```

Otras soluciones válidas

- Probar funciones puras con datos controlados.
- Añadir una prueba del error esperado para lista vacía.

6. Fuentes oficiales

Esta guía resume conceptos para estudiar. Consulta las fuentes para comprobar detalles de versión y APIs.

Node.js — Test runner

Node.js — Assertion testing